

Question Numbers

1 2 3 4 5
6

6 / 6

100 %

Berikut ini adalah *schedule* hasil eksekusi 3 buah transaksi secara konkuren. Pada *schedule* Ry(A) dan Wy(A) menyatakan operasi yang dilakukan oleh transaksi Ty terhadap item data A, Cy menyatakan commit transaksi Ty.
R1(X); R3(Y); R3(X); W3(Y); R2(Y); R1(Y); W3(X); R2(X); C1; C2; C3;

Sebutkan apakah *schedule* tersebut: **serializable**, **recoverable**, dan **cascadeless** dan jelaskan.

Paragraph

Precedence graph:
X: $T1 \rightarrow T3 \rightarrow T2$
Y: $T3 \rightarrow T1 \mid T2$

Ada cycle $T1 \rightarrow T3 \rightarrow T1$ sehingga *schedule* tidak serializable.

Schedule juga tidak recoverable, karena T1 membaca data Y yang ditulis oleh T3 tetapi T1 melakukan commit sebelum T3 melakukan commit.

Schedule tidak cascadeless, karena T1 dan T2 membaca data yang ditulis oleh T3 sebelum T3 melakukan commit. Hal ini menyebabkan terjadinya dirty read.

[Save Answer](#)

Question Numbers

1 2 3 4 5
6

6 / 6

100 %

Berikut ini adalah *schedule* hasil eksekusi 3 buah transaksi secara konkuren. Pada *schedule* Ry(A) dan Wy(A) menyatakan operasi yang dilakukan oleh transaksi Ty terhadap item data A, Cy menyatakan commit transaksi Ty.
R1(X); R2(Z); R1(Z); R3(X); R3(Y); W1(X); C1; W3(Y); C3; R2(Y); W2(Z); W2(Y); C2;

Sebutkan apakah *schedule* tersebut: **serializable**, **recoverable**, dan **cascadeless** dan jelaskan.

Paragraph

Precedence graph:
X: $T3 \rightarrow T1$
Y: $T3 \rightarrow T2$
Z: $T1 \rightarrow T2$

Schedule serializable karena precedence graph tidak mengandung cycle

Schedule cascadeless karena T2 membaca data Y yang ditulis oleh T3 setelah T3 melakukan commit.

Karena setiap *schedule* yang cascadeless pasti recoverable, maka *schedule* ini juga recoverable.

[Save Answer](#)

Question Numbers

1 2 3 4 5
6

6 / 6

100 %

Berikut ini adalah *schedule* hasil eksekusi 3 buah transaksi secara konkuren. Pada *schedule* Ry(A) dan Wy(A) menyatakan operasi yang dilakukan oleh transaksi Ty terhadap item data A, Cy menyatakan commit transaksi Ty.

R1(X); R2(Z); R1(Z); R3(X); R3(Y); W1(X); W3(Y); R2(Y); W2(Z); W2(Y); C1; C2; C3;

Sebutkan apakah *schedule* tersebut: **serializable**, **recoverable**, dan **cascadeless** dan jelaskan.

Paragraph 

Precedence graph:

X: T3→T1

Y: T3→T2

Z: T1→T2

Tidak terdapat cycle pada precedence graph, sehingga *schedule* serializable.

Schedule tidak recoverable, karena T2 membaca Y yang ditulis oleh T3 sebelum T3 commit, tetapi C2 terjadi sebelum C3.

Schedule tidak cascadeless, karena T2 melakukan read(Y) setelah T3 melakukan Write(Y), tetapi sebelum T3 melakukan commit (artinya, T2 membaca data yang belum dicommit (dirty read))

Save Answer

Question Numbers

1 2 3 4 5
6

6 / 6

100 %

Berikut ini adalah *schedule* hasil eksekusi 2 buah transaksi secara konkuren. Pada *schedule* Ry(A) dan Wy(A) menyatakan operasi yang dilakukan oleh transaksi Ty terhadap item data A, Cy menyatakan commit transaksi Ty.

R1(Z); W1(Y); R2(Y); W2(X); W1(X); W2(X); C1; C2;

Sebutkan apakah *schedule* tersebut: **serializable**, **recoverable**, dan **cascadeless** dan jelaskan.

Paragraph 

Precedence graphL

X: T2 → T1 → T2

Y: T1 → T2

Z: T1

Precedence graph mengandung cycle T2 → T1 → T2 sehingga *schedule* tidak conflict serializable. Namun, *schedule* view serializable karena ekuivalen dengan serial order T1 → T2: R2(Y) membaca nilai Y yang ditulis oleh T1. Lalu final write pada X dilakukan oleh T2, sama seperti pada serial order T1 → T2.

Schedule recoverable karena T2 membaca data yang ditulis oleh T1 dan C1 terjadi sebelum C2.

Schedule tidak cascadeless, karena R2(Y) membaca data yang ditulis oleh T1 sebelum T1 melakukan commit (dirty read)

Save Answer

Question Numbers

1 2 3 4 5 6

6 / 6

100 %

Di antara masalah yang harus dicegah saat eksekusi transaksi adalah: (1) *dirty read*, (2) *lost update*, (3) *transaction failure*, dan (4) *inconsistent database state*. Apabila dua transaksi T1 dan T2 yang berjalan secara konkuren dan melakukan perubahan terhadap item data yang sama dieksekusi tanpa menggunakan mekanisme kontrol konkurensi, maka masalah yang dapat muncul adalah:

- ☐ (1), (2), dan (3)
- ☐ (3) dan (4)
- ☐ (1) dan (2)
- ☒ (1), (2), dan (4)

[← Back](#)[Next Question →](#)

Question Numbers

1 2 3 4 5 6

6 / 6

100 %

Pernyataan tentang level konsistensi yang didefinisikan pada SQL-92 berikut ini benar, kecuali:

- ☒ *Serializable*: seluruh transaksi dijalankan secara serial untuk memastikan terjaganya properti *isolation*.
- ☐ *Repeatable read*: hanya data yang sudah *committed* yang dapat dibaca dan hasil pembacaan sebuah item selalu sama. Memungkinkan terjadinya *phantom phenomena*.
- ☐ *Read committed*: hanya data yang sudah *committed* yang dapat dibaca. Jika sebuah item data dibaca beberapa kali, nilai yang diperoleh mungkin tidak sama.
- ☐ *Read uncommitted*: data yang belum *committed* tetap boleh dibaca.

[← Back](#)[Finish Exam →](#)